

2026 KAIST RUN Spring Contest Editorial

Official Editorial

by
RUN

문제	의도한 난이도	출제자
A Second Run	Easy	ibm2006
B RUN Sequence	Medium	mon_de2738
C Dubai Chewy Cookie	Medium	ibm2006
D Grilled Bottle	Medium	mon_de2738
E Kirameki of Revue	Hard	gubshig
F Pragmatism	Hard	ibm2006
G AiScReam	Hard	ibm2006
H Hawaiki	Challenging	gmroh06
I Wizdomiot	Challenging	ibm2006

A. Second Run

constructive

출제진 의도 – **Easy**

- 처음 푼 사람: **TLE**, 4분
- 출제자: **ibm2006**

A. Second Run

- $i = 1, 2, \dots, N$ 에 대해서, $w_j \leq i$ 인 요구들 중 만족된 것의 개수가 $1/4$ 이상이 되도록 A_i 를 결정할 수 있습니다.
- $w_j = i$ 인 어떤 요구에 대해서, 조건을 만족하도록 하는 A_i 는 정확히 한 가지 존재합니다.
- 비둘기집의 원리에 의해서 어떤 A_i 는 $w_j = i$ 인 요구들 중 $1/4$ 이상을 만족하도록 할 수 있습니다.
- 따라서 $i = 1, 2, \dots, N$ 에 대해서 그러한 A_i 들을 선택해주면 됩니다.

A. Second Run

- 랜덤 알고리즘으로도 문제를 해결할 수 있습니다.
- A_i 들을 독립적으로 랜덤하게 결정하면, 각 요구가 만족될 확률은 정확히 $1/4$ 입니다.
- 따라서 각 A_i 들을 랜덤하게 결정해주면 유의미한 확률로 전체 요구중 $1/4$ 이상이 만족됩니다.
- 이러한 랜덤 할당을, 전체 요구중 $1/4$ 이상이 만족될 때까지 시도하면 됩니다.

B. RUN Sequence

math

출제진 의도 – **Medium**

- 처음 폰 사람: **azber**, 18분
- 출제자: mon_de2738

B. RUN Sequence

- F_n 을 피보나치 수열이라 할 때, $a_N = F_{N-2}a_1 + F_{N-1}a_2$ 임을 알 수 있습니다.
- $\gcd(F_{N-1}, F_{N-2}) = 1$ 이므로, a_1 이 F_{N-1} 만큼 감소하고, a_2 가 F_{N-2} 만큼 증가할 때 기존의 a_N 값과 겹치게 됩니다.
- 겹치는 횟수는 $\max(0, R - F_{N-1}) \times \max(0, U - F_{N-2})$ 으로 계산할 수 있습니다.
- $R \times U$ 에서 위의 식을 뺀 값이 답이 됩니다.

C. Dubai Chewy Cookie

dynamic programming, probability theory

출제진 의도 – **Medium**

- 처음 푼 사람: **undertaker**, 34분
- 출제자: **ibm2006**

C. Dubai Chewy Cookie

- ”두바이 쫄득 쿠키”는 다소 긴 표현이므로, 앞으로의 에디토리얼에서는 ”두쫄쿠”로 줄여 말하겠습니다.

C. Dubai Chewy Cookie

- 어떤 체인점 v 가 재료를 얻었을 때 1번 체인점이 두쫂꾸 한 상자를 받고, 2번 체인점은 받지 못한다고 합시다. 그러한 체인점들로 구성된 집합 A 를 생각합니다.
- 비슷하게 2번 체인점만 상자를 받도록 하는 체인점들의 집합 B , 1, 2번 체인점 둘 다 받도록 하는 체인점들의 집합 C 를 생각합니다.

C. Dubai Chewy Cookie

- 1 번 체인점이 s 상자의 두쫂꾸를 받고, 2 번 체인점이 t 상자의 두쫂꾸를 받았다는 것은 어떤 0 이상 $\min(s, t)$ 이하의 정수 k 가 존재하여, 집합 A, B, C 에서 각각 정확히 $s - k, t - k, k$ 개의 체인점이 재료를 구했다는 것을 의미합니다.
- $DP[X][n]$ 를, 집합 X 에 속한 체인점 중 정확히 n 개의 체인점들이 재료를 구할 확률이라고 정의하겠습니다. 그렇다면 답은 다음과 같이 나타납니다:

$$\sum_{k=0}^{\min(s,t)} DP[A][s-k]DP[B][t-k]DP[C][k]$$

C. Dubai Chewy Cookie

- 각 DP 배열은 적절한 다이나믹 프로그래밍을 통해 $O(N^2)$ 에 구해질 수 있습니다.
- DP 배열을 구했다면, 각 쿼리는 $O(\min(s, t)) = O(N)$ 에 구해질 수 있습니다.
- 따라서 전체 문제는 $O(M + N^2 + QN)$ 에 해결될 수 있습니다.

D. Grilled Bottle

greedy, binary search

출제진 의도 – Medium

- 처음 푼 사람: **newbie**, 13분
- 출제자: mon_de2738

D. Grilled Bottle

- 만약 구울 병의 개수를 미리 정해 두었다면 어떤 전략이 최적일까요?
- 요구되는 화력이 가장 큰 병부터, 그 병을 구울 수 있는 가장 만족도가 큰 불로 구우면 됩니다.
- 또한 x 개의 병을 굽는 방법이 존재한다면, $x - 1$ 개의 병을 굽는 방법 또한 존재합니다.
- 따라서 구울 병의 개수를 기준으로 이분 탐색을 하면 $\mathcal{O}((M + N) \log(M + N))$ 이나 $\mathcal{O}((M + N) \log^2(M + N))$ 의 시간 복잡도로 문제를 해결할 수 있습니다.
- 생각해 보면, 만족도를 최대화하기 위해선 굽는 병의 개수도 최대화돼야 한다는 사실 또한 알 수 있습니다.

E. Kirameki of Revue

Parametric Search, Trie

출제진 의도 – **Hard**

- 처음 푼 사람: **undertaker**, 60분
- 출제자: gubshig

E. Kirameki of Revue

- 반짝임이 X 이하인 리뷰의 개수를 세봅시다.
- 이는 X 가 고정일때 Trie를 사용해 $\mathcal{O}(N \log X)$ 에 해결할 수 있습니다.
- 파라메트릭 서치로 문제를 $\mathcal{O}(N \log^2 X)$ 에 풀 수 있습니다.

F. Pragmatism

constructive, ad hoc, DFS tree

출제진 의도 – **Hard**

- 처음 푼 사람: **TLE**, 107분
- 출제자: ibm2006

F. Pragmatism

- 아무 순서대로 DFS를 합시다.

F. Pragmatism

- 아직 한 번도 방문하지 않은 정점의 집합을 S , 모든 탐색이 끝난 정점의 집합을 T 라고 하겠습니다.
- DFS 과정을 시작하게 전, S 는 모든 정점들의 집합이고 T 는 비어있습니다.
- DFS가 끝난 뒤, S 는 비어있고 T 는 모든 정점들의 집합입니다.

F. Pragmatism

- 한 번의 DFS 이동 과정에서, S 와 T 중 최대 하나의 크기가 최대 1 만큼 바뀝니다.
- 따라서 $|S| = |T|$ 인 순간이 존재합니다.
- S 와 T 사이에 간선이 존재할 수 없습니다. 간선이 존재했다면 T 의 탐색을 끝내기 이전에 S 의 탐색이 시작되어야 했기 때문입니다.
- 따라서 S, T 는 대립의 신전을 이룹니다.

F. Pragmatism

- S, T 중 어느것에도 속하지 않은 정점들은 경로를 이룹니다. 즉, 이들은 빛의 신전을 이룹니다.
- $|S| = |T|$ 인 시점, 이 빛의 신전과 앞서 말한 대립의 신전의 크기의 합은 N 입니다.
- 만약 이 순간 빛의 신전의 크기가 $N - 2K + 2$ 미만이고, 대립의 신전의 크기가 $2K$ 미만이라면, 두 신전의 크기의 합이 N 미만이어야 하므로, 이는 모순입니다.
- 따라서 둘 중 하나는 문제의 조건을 만족합니다.

G. AiScReam

ad hoc, divide and conquer

출제진 의도 – **Hard**

- 처음 푼 사람: **null(ptr)scapes**, 152분
- 출제자: **ibm2006**

- 분할정복을 통해 해결해봅시다.
- 편의상, 구간 $[x, y]$ 에 있는 모든 아이스크림을 i 번 멤버가 전부 먹었을 때 얻는 만족감을 $eval(i, x, y)$ 와 같이 표기하고, i 번 멤버가 $[x, y]$ 에 있는 아이스크림을 먹었을 때 얻는 만족감이 r 이 되는 y 의 값을 $cut(i, x, y)$ 와 같이 표기하겠습니다.
- 구간 $[L, R]$ 에 대해서 $S = \{i_1, i_2, \dots, i_k\}$ 번 멤버들 각자에게 적어도 $\frac{eval(i_j, L, R)}{k}$ 만큼의 만족감을 얻도록 분배하는 함수 $Solve(L, R, S)$ 를 생각합시다. 이때, k 는 2의 거듭제곱입니다.

- $k = 1$ 이면 해당 멤버가 자명하게 $[L, R]$ 사이의 모든 아이스크림을 먹으면 됩니다.
- $k = 2$ 라면, 다음과 같은 multiset $T = \{cut(x, L, \frac{eval(x, L, R)}{2}) | x \in S\}$ 를 생각합시다. 즉, 각 멤버들이 생각하기에 만족감이 정확히 반으로 잘리는 지점들을 모아둔 multiset을 생각합니다.
- T 의 $\frac{k}{2}$ 번째 값을 mid 라고 하겠습니다.

- S 를 크기가 같은 두 집합 A, B 로 나눠서, 모든 $x \in A$ 에 대해서 $cut(x, L, \frac{eval(x, L, R)}{2}) \leq mid$ 가 성립하고, 모든 $x \in B$ 에 대해서 $cut(x, L, \frac{eval(x, L, R)}{2}) \geq mid$ 가 성립하도록 할 수 있습니다.
- 따라서 $Solve(L, mid, A), Solve(mid, R, B)$ 를 호출하여 문제를 해결할 수 있습니다.
- A 에 있는 모든 멤버들에 대해서, $[L, mid]$ 에 있는 모든 아이스크림의 만족도의 총합은 $[L, R]$ 에 있는 모든 아이스크림의 만족도의 총합의 절반보다 크거나 같고, B 에 대해서도 유사한 사실이 성립하기 때문입니다.

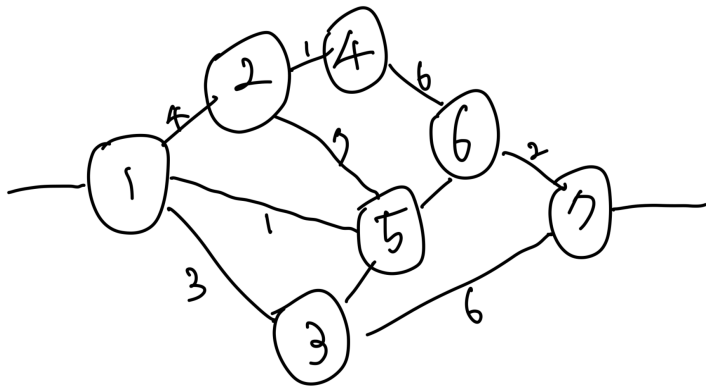
- 해당 문제에서는 정확한 채점과 간결한 구현을 위해서 $\epsilon = \frac{1}{2^{20}}$ 의 오차를 허용하였고, 문제 조건을 2의 거듭제곱으로 조정하였습니다.

H. Hawaiki

flow, planar_graph, dual_graph

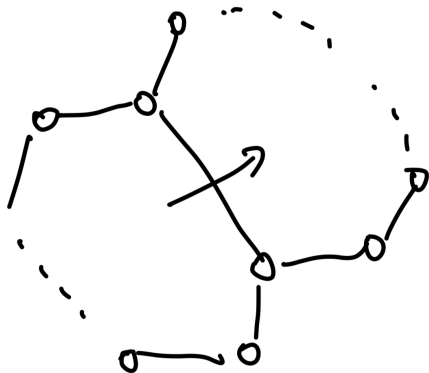
출제진 의도 – **Challenging**

- 처음 푼 사람: **TLE**, 191분
- 출제자: gmroh06



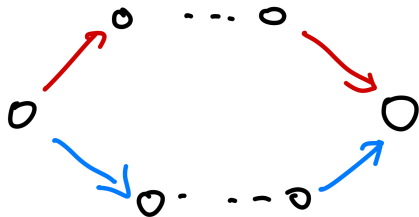
- 1 번 섬에서 $-x$ 축 방향으로, N 번 섬에서 $+x$ 방향으로 반직선을 긁습니다.
- 이제 문제를 듀얼 그래프에서 생각해 보겠습니다.

H. Hawaiki

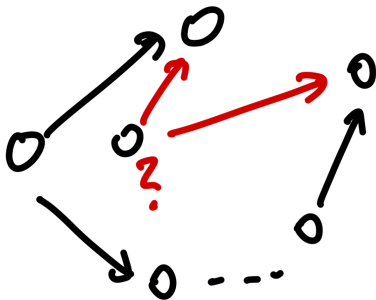


- 모든 정점의 x 좌표가 다르기 때문에 면 사이의 간선을 아래에서 위로 가는 가중치가 같은 방향 듀얼 간선으로 생각할 수 있습니다.
- 이때, 방향성을 추가한 듀얼 그래프는 DAG가 됩니다.

H. Hawaiki



- 방향 듀얼 그래프가 DAG임은 올바른 위상 정렬 방법이 존재함을 보여 증명할 수 있습니다.
- 하나의 면을 둘러싸는 간선들을 x 좌표가 최소인 정점과 최대인 정점을 기준으로 2개로 분할합니다.
- 앞으로 이 분할을 각각 위쪽 간선, 아래쪽 간선으로 부르겠습니다.
- x 좌표가 최소인 정점에서 최대인 정점 방향으로의 탐색을 생각해 보면 간선들은 반드시 x 좌표가 증가해야 합니다.



- 만약 역전이 발생한다면 $W_j > 0$ 이며 모든 간선의 조건을 만족시키는 배치가 존재한다는 것에 위배됩니다.

H. Hawaiki

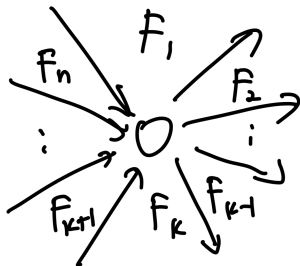
- 모든 간선을 출발 정점 기준으로 반시계 방향 각도 정렬합니다.
- 간선들을 정렬된 순서로 1번 정점부터 DFS하면서 하나씩 추가한다고 생각해 보겠습니다.
- 어떤 면을 둘러싸는 모든 간선이 추가되는 순서로 모든 면을 정렬할 수 있습니다.
- DFS의 성질에 의해 위쪽 간선은 아래쪽 간선들이 모두 추가된 후에 추가됩니다.
- 면 f 가 방금 완성된 상태라고 하면, 방향 듀얼 그래프에서 f 로 들어오는 것은 아래쪽 간선을 통해서만 가능합니다.
- f 의 아래쪽 간선이 모두 추가된 상태이며, f 의 아래쪽 간선들은 아래쪽 면들의 입장에서 위쪽 간선이기 때문에 DFS의 성질에 의해 아래쪽 면들은 모두 완성된 상태입니다.
- f 로 들어올 수 있는 모든 면들이 정렬 순서 상 f 의 앞에 있으므로, 이 위상 정렬은 올바릅니다.
- 따라서, 방향 듀얼 그래프는 DAG입니다. (Lemma 1)

H. Hawaiki

- 듀얼 DAG 상에서 어떤 경로가 있을 때, 그 경로를 이루는 듀얼 간선에 대응되는 간선의 집합을 P 라고 하겠습니다.
- 어떤 행해사의 $1 \rightarrow N$ 경로를 이루는 간선의 집합을 Q 라 하면, $|P \cap Q| \leq 1$ 입니다.
- Q 에 속한 모든 간선은 해당 간선 기준 아래쪽 면에서 위쪽 면으로 가는 듀얼 간선을 만듭니다.
- Q 가 나누는 두 면 집합을 생각하고, 아래쪽 집합을 S , 위쪽 집합을 T 라 하겠습니다.
- S 와 T 사이 모든 간선은 S 에서 T 로 가므로, DAG의 성질에 의해 한 번 S 에서 T 로 넘어가면 다시 돌아오는 것은 불가능합니다.
- 따라서, $|P \cap Q| \leq 1$ 입니다. (Lemma 2)

H. Hawaiki

- Lemma 1에 의해 듀얼 그래프에서 최장 경로가 존재합니다.
- Lemma 2에 의해 이 최장 경로의 길이는 자명한 하한입니다.
- 어떤 면 f 으로 가는 듀얼 DAG 상의 최장 경로를 D_f 라 정의합니다.
- 어떤 간선 e 기준으로 아래쪽 면을 x , 위쪽 면을 y 라 합니다.
- e 를 지나는 항해사 수를 $D_y - D_x$ 로 결정합니다.
- 최장 경로의 정의에 의해 $D_x + W_e \leq D_y$ 이므로, $W_e \leq D_y - D_x$ 를 만족합니다.



- 상술한 방법이 플로우의 조건을 만족하는지 확인합니다.
- 어떤 정점 v 를 둘러싼 면을 시계 방향으로 $F_1, \dots, F_k, \dots, F_n$ 으로 하겠습니다.
- v 로 들어오는 항해사 수는

$$(D_{F_1} - D_{F_n}) + (D_{F_n} - D_{F_{n-1}}) + \dots + (D_{F_{k+1}} - D_{F_k}) = D_{F_1} - D_{F_k}$$
 입니다.
- 마찬가지로 v 에서 나가는 항해사 수도 $D_{F_1} - D_{F_k}$ 이므로 플로우의 조건을 만족합니다.

H. Hawaiki



- 자명한 하한이 실제로 구성 가능함을 보였으므로 최적의 정답이 됩니다.
- 구현은 Lemma 1을 증명할 때 사용한 접근법을 이용하면 편리합니다.

I. Wizdomiot

data structure, segment tree, combinatorics, divide and conquer

출제진 의도 – **Challenging**

- 제출 X번, 정답 X명 (정답률 X%)
- 처음 푼 사람: **asdf**, 분
- 출제자: ibm2006

- 전체 문제는, 간단한 더블카운팅을 통해 다음과 같은 문제로 환원할 수 있습니다.
- 길이 n 의 배열 A 가 있다. A 의 모든 원소는 처음에는 0이다. 총 n 개의 쿼리가 주어지는데, 이중 i 번째 쿼리는 A 의 x_i 번째 원소를 1로 수정하라는 것이다. 각 쿼리를 시행한 뒤, 모든 $1 \leq L \leq R \leq n$ 에 대해서 $[L, R]$ 에 속한 가장 긴 연속한 1로 구성된 부분배열의 길이를 전부 합한 값을 알아내라.

I. Wizdomiot

- 배열 A 에 대한 세그먼트 트리를 만들고 나면, 해당 세그먼트 트리의 각 노드를 표현하는 구간 $[s, e]$ 에 대해서 $s \leq L \leq \frac{s+e}{2} < R \leq e$ 인 L, R 의 쌍에 대해서만 위의 문제를 해결하면 충분합니다.
- 한 번의 쿼리가 최대 $O(\log n)$ 개의 노드를 변경하므로, 위의 문제를 빠르게 해결할 수 있으면 전체 문제를 해결 가능합니다.

I. Wizdomiot

- 이제, 전체 문제는 아주 대략적으로 0을 1로 바꾸는 쿼리들에 대해서,
 $1 \leq L \leq \frac{n}{2} = mid < R \leq n$ 인 모든 L, R 에 대해 $[L, R]$ 에 존재하는 가장 긴 1로 구성된 부분배열의 길이를 전부 합한 값을 알아내는 것으로 환원되었습니다.
- 이를 해결하기 위해, L, R 의 위치에 따라 케이스를 3가지로 나눕니다.
- $[lmax, mid]$ 가 1로만 구성된 구간이도록 하는 가장 작은 $lmax$ 를 생각하고, $[mid + 1, rmax]$ 가 1로만 구성된 구간이도록 하는 가장 큰 $rmax$ 를 생각합시다. $lmax, rmax$ 는 적절한 자료구조 혹은 관찰을 통해 쉽게 관리될 수 있음을 관찰합시다.

I. Wizdomiot

- Case 1: $lmax \leq L, R \leq rmax$
- 이 경우는 간단한 계산을 통해 $O(1)$ 에 계산 가능합니다.

I. Wizdomiot

- Case 2: $lmax \leq L, R > rmax$
- 여기에서, $B[i] = ([mid+1, mid+1+i]$ 상에서 가장 긴 1로 구성된 부분배열의 길이)를 생각하겠습니다. 추후에 설명하겠지만, 지금은 B 가 대부분의 난해한 쿼리를 처리할 수 있는 만능의 자료구조 위에서 관리되는 경우를 생각하겠습니다.
- 모든 R 에 대해서 각각 답을 계산한 다음 합치는 방식으로 문제를 해결할 것입니다.
- 고정된 $R = r$ 에 대해서, 각 $lmax \leq L \leq mid$ 에 대한 최대길이는 $max(B[r], rmax - L + 1)$ 이 됩니다.
- 해당 값의, L 에 대한 전체합은 $B[r]$ 과 $rmax - mid + 1, rmax - lmax + 1$ 사이의 대소에 따라 달라집니다. 이로부터 3개의 서브케이스가 발생합니다.

I. Wizdomiot

- Case 2-1: $B[r] \leq rmax - mid + 1$.
- 이 경우 모든 L 의 값에 대해서 $B[r] \leq rmax - L + 1$ 이므로, 수학적으로 $O(1)$ 에 모든 합을 구할 수 있습니다.
- Case 2-2: $B[r] \geq rmax - lmax + 1$.
- 이 경우 항상 $B[r] \geq rmax - L + 1$ 이므로, 답은 단순히 $B[r] \times (mid - lmax + 1)$ 이 됩니다.
- Case 2-3: otherwise
- 이 경우 Case 2-1에서, 대략적으로 $\binom{B[r] - (rmax - mid + 1)}{2}$ 에 비례하는 항을 더하여 보정해줄 수 있습니다.

- $B[r]$ 은 단조적이므로, 각 케이스에 대응되는 구간은 총 3개로 나뉩니다. 2-1번 케이스인 구간에 대해서는 구간의 길이만이 필요하며, 2-2번 케이스인 구간에 대해서는 구간에 존재하는 $B[r]$ 의 합만이 필요하며, 2-3번 케이스인 구간에 대해서는 구간에 존재하는 $B[r]$ 과 그 제곱의 합을 얻으면 모든 케이스에 대한 답의 합을 적절히 계산 가능합니다.
- 물론, $L < lmax, R \leq rmax$ 인 경우 또한 대칭적으로 해결해줄 수 있습니다.

- Case 3: $L < lmax, rmax < R$.
- 위의 B 와 유사하게, $A[i] = ([mid - i, mid]$ 상에서 가장 긴 1로 구성된 부분배열의 길이)와 같이 정의하겠습니다.
- 이 경우 고정된 L, R 에 대한 답은 $\max(B[R], A[L], rmax - lmax + 1)$ 이 됩니다.
- 더하는 관점을 바꿔봅시다. 모든 L, R 에 대해 구간의 길이를 더하는 대신, 구간의 길이 t 에 대해서 $ans(L, R) \geq t$ 인 L, R 의 가짓수를 모든 $t \geq 1$ 에 대해서 더해도 됩니다.

I. Wizdomiot

- $invA[i] =$
([mid-L,mid]의 가장 긴 1로만 구성된 부분배열의 길이가 i 미만이도록 하는 최대의 L) 과 같이 정의하겠습니다. 유사하게 $invB$ 를 정의할 수 있습니다.
- 그렇다면 답은
$$mid \times (n - mid) \times (rmax - lmax + 1) + \sum_{t \geq rmax - lmax + 2} (mid \times (n - mid) - invA[t] invB[t])$$
임을 관찰할 수 있습니다.
- $invA, invB$ 를 "잘" 관리한다면 위의 구간합을 적절히 관리할 수 있습니다.

I. Wizdomiot

- 위의 각 케이스들에 대한 알고리즘의 구현을 매우 현명하고 효율적으로 해낼 수 있다면 전체 문제가 해결됨을 알 수 있습니다.
- 구현 디테일이 몹시 복잡한 관계로 지금까지의 설명은 몹시 대략적인 점 양해 부탁드립니다.
- 이제부터 $A, B, invA, invB$ 를 관리하는 자료구조에 대해 설명하겠습니다.

I. Wizdomiot

- B 와 $invB$ 는 "대략적으로" 2차원 평면 상에서 $x = y$ 에 대한 대칭 관계임을 관찰할 수 있습니다.
- B 에 대응되는 수열에서, 어떤 0을 1로 바꾸면 B 에서는 평행사변형 모양의 업데이트가 여러 차례 발생할 수 있고, $invB$ 에서는 직사각형 모양의 업데이트가 여러 차례 발생할 수 있습니다.
- 특정한 이유로 인해, $invB$ 자체를 관리하는 것 보다는 $invB - i$ 를 관리하는 것이 더욱 간단합니다. B 에서 발생하는 각 평행사변형 업데이트가 $invB - i$ 에서는 정확히 같은 값을 가지는 구간에 일정한 값을 더하는 쿼리로 환원되기 때문입니다.

I. Wizdomiot

- 이러한 관찰을 기반으로 실제로 발생해야 할 업데이트는 $invB - i$ 내부에서 관리하는 것이 간편합니다. 어떤 1이 추가되는 업데이트는 $invB - i$ 에 대해서 "구간 $[1, X]$ 에 존재하는 모든 값에 $\min(\cdot, Y)$ 를 취하여라"와 같은 형식의 쿼리로 변환될 수 있습니다.
- 이러한 쿼리는 해당 구간에서 가장 큰 값을 가지는 구간에서 연속적으로 일정한 값을 빼주는 연산을 수 차례 반복하여 처리해줄 수 있습니다.
- 해당 연산이 한 번의 쿼리에서 여러번 발생할 수 있지만, 전체 연산은 amortized하게 $O(n)$ 임을 직관적으로 알 수 있습니다.

I. Wizdomiot

- 따라서 해당 쿼리를 처리해주면서 $invB - i, invB$ 에 대응되는 배열에는 구간 업데이트 적용해주고, B 에는 "평행사변형 업데이트"를 적용해주면 됩니다. 비슷하게 $invA - i, invA, A$ 도 관리 가능합니다.
- 구간 업데이트를 지원하며 $invB \times invA$ 의 구간 합을 처리하는 것은 세그먼트 트리를 통해 관리 가능합니다.
- B 에 대해서는 평행사변형 업데이트를 하며 구간에 대한 원소의 합과 제곱합을 관리해야 하는데, 이는 (복잡한) 세그먼트 트리를 통해 관리 가능합니다.